

<mutex> Header in C++ - Key Components & Examples

Overview

The <mutex> header provides synchronization primitives for safe concurrent programming. It mainly helps protect shared data by allowing only one thread at a time to access a resource.

Key Components

- `std::mutex` - Basic mutex lock
- `std::recursive_mutex` - Mutex lockable multiple times by same thread
- `std::timed_mutex` - Mutex with timeout capability
- `std::recursive_timed_mutex` - Recursive mutex with timeout
- `std::lock_guard<Mutex>` - RAII wrapper for mutex locking
- `std::unique_lock<Mutex>` - Flexible RAII mutex wrapper
- `std::try_lock` - Tries to lock multiple mutexes without deadlock
- `std::once_flag` - Used with `call_once` for one-time initialization
- `std::call_once` - Ensures function called once thread-safely

Important Concepts

- Mutex: Provides exclusive ownership; only one thread can lock at a time.
- Lock Guard: Locks mutex on creation, unlocks on destruction (scope-based).
- Unique Lock: Allows manual lock/unlock control and deferred locking.
- Recursive Mutex: Same thread can lock multiple times without deadlock.
- Timed Mutex: Allows lock attempts with timeout.
- Once Flag + `call_once`: Thread-safe one-time initialization.

Common Usage Examples

Basic Mutex + Lock Guard:

<mutex> Header in C++ - Key Components & Examples

```
#include <iostream>
```

```
#include <thread>
```

```
#include <mutex>
```

```
std::mutex mtx;
```

```
int counter = 0;
```

```
void increment() {
```

```
    for (int i = 0; i < 10000; ++i) {
```

```
        std::lock_guard<std::mutex> lock(mtx);
```

```
        ++counter;
```

```
    }
```

```
}
```

```
int main() {
```

```
    std::thread t1(increment);
```

```
    std::thread t2(increment);
```

```
    t1.join();
```

```
    t2.join();
```

```
    std::cout << "Counter: " << counter << "\n";
```

```
}
```

Unique Lock with Manual Unlock:

```
#include <mutex>
```

```
std::mutex mtx;
```

```
void do_work() {
```

```
    std::unique_lock<std::mutex> lock(mtx);
```

<mutex> Header in C++ - Key Components & Examples

```
// Critical section
lock.unlock(); // unlock early

// Non-critical work

lock.lock(); // re-lock if needed
}
```

call_once for One-Time Initialization:

```
#include <iostream>
```

```
#include <mutex>
```

```
std::once_flag flag;
```

```
void init() {
    std::cout << "Initialized once\n";
}
```

```
void worker() {
    std::call_once(flag, init);
    std::cout << "Worker running\n";
}
```

```
int main() {
    std::thread t1(worker);
    std::thread t2(worker);

    t1.join();
    t2.join();
}
```